INTERNAL PROPOSAL · INFRASTRUCTURE & BUDGET

The case for routing our own inference

Analysing 1,800+ ministry transcripts is not a hard engineering problem — it is a procurement problem. The pipeline is already built and already provider-neutral. The only remaining decision is which model we pay to run it, and that single choice moves the cost of a full corpus pass by a factor of roughly **175×**.

Prepared for Foundation leadership **Subject** Adopting OpenRouter as the inference provider

Date 27 August 2026 **System** transcript-theme-analyzer

THE RECOMMENDATION IN FOUR NUMBERS

$227
One full pass over the corpus on Claude Opus 5

$1.29
The identical pass on DeepSeek V4 Flash via OpenRouter

0
Lines of code that must change to switch

$2,713
Saved on a twelve-theme research programme

01

What we are actually asking a model to do

Before comparing prices, the workload has to be stated precisely — because the shape of the task is the reason a cheaper model is defensible here.

The analyzer takes a transcript and a theme — say *“the glory of God”* — and returns a structured judgement: a relevance score from 0 to 100, whether the treatment is explicit or tangential, a short reasoning paragraph, and a list of passages where the theme is discussed. Each passage is identified not by quoting it, but by returning a short verbatim phrase at its start and another at its end.

That last detail matters more than it looks. The model never reproduces the passage text. It returns two boundary markers, and `extraction.py` then locates and slices the real passage out of the source with a regular expression. The expensive part of the job — emitting long verbatim quotations — was deliberately designed out of the model's responsibilities and handed to deterministic code.

Transcripts under the single-pass limit are handled in one call. Longer ones are split into overlapping ~12,000-token chunks, scored in parallel, and reconciled by one final synthesis call. So the unit of work is not “read a transcript” — it is a small number of tightly-constrained, schema-bound scoring calls.

> WHY THIS FRAMING DECIDES THE ARGUMENT
>
> We are not asking a model to write, to reason across documents, to use tools, or to hold a long agentic conversation. We are asking it to read one bounded passage, apply a rubric, and fill in a fixed JSON schema. This is the single most commoditised capability in the current model market — the task where the gap between a frontier model and a competent cheap one is narrowest.

The measured shape of one transcript

These are not estimates. They are the recorded usage from a real run already sitting in `results/`, for one chunked transcript:

Recorded token usage, one chunked transcript

Source: `results/x-ai__grok-4.3.json` — 5 calls, 14.6 seconds wall clock

CALL	INPUT TOKENS	OUTPUT TOKENS
chunk_0	11,240	1,059

CALL	INPUT TOKENS	OUTPUT TOKENS
chunk_1	11,149	929
chunk_2	11,118	881
chunk_3	2,001	922
synthesis	769	431
Total	**36,277**	**4,222**

Note the ratio: **input tokens outnumber output tokens roughly nine to one**. This workload is overwhelmingly about *reading*, not *writing*. That is important because output tokens are the expensive half of every price list — typically four to five times the input rate. A read-heavy workload is exactly the profile where a cheap model's advantage compounds hardest.

Scaling that to the corpus

Extrapolating across 1,800 transcripts with a realistic length mix — 30% short (~30 min), 45% medium (~1 hr), 25% long (~2 hr) — and reproducing the pipeline's own single-pass/chunking branch gives the workload below. The model that produces these figures is checked into the repository as `cost_model.py`, so any of these numbers can be re-derived — and re-run against the real corpus with `--transcript-dir` once it is mounted, for exact rather than modelled figures.

One full corpus pass, one theme

1,800 transcripts, single model, no retries

QUANTITY	VOLUME
API calls	3,150
Input tokens	31,011,750
Output tokens	2,893,500
Total tokens	**33,905,250**

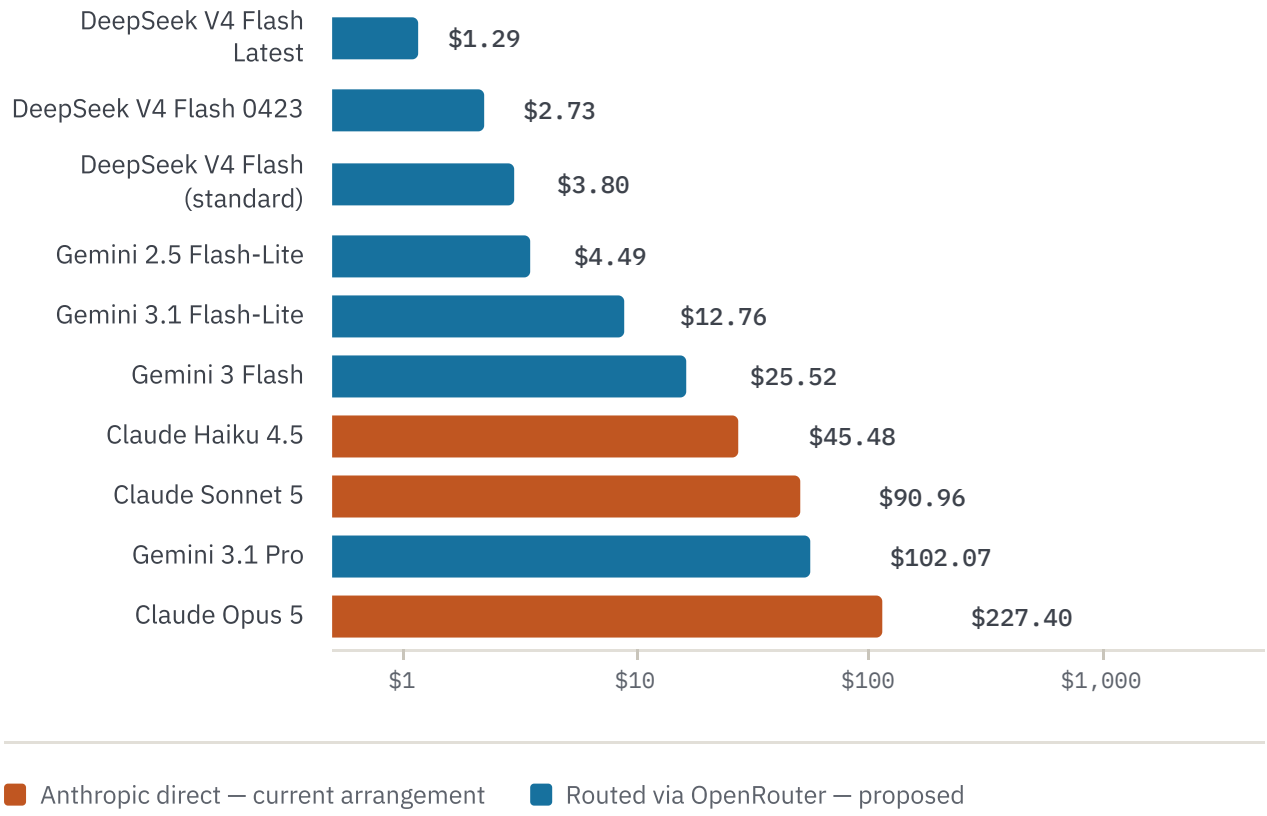
02

What that workload costs, by model

Same tokens, same pipeline, same output schema. The only variable is the price list.

Cost of one full corpus pass

1,800 transcripts × 1 theme · logarithmic scale — the range spans three orders of magnitude, so bar length shows scale and the printed figure beside each bar carries the exact value. The full table below gives the same figures with per-transcript cost and token rates.



Full price comparison

List rates per million tokens, fetched 27 Aug 2026. OpenRouter figures include the 5.5% credit top-up fee; OpenRouter applies no per-token markup.

MODEL	ROUTE	IN $/M	OUT $/M	CORPUS PASS	PER TRANSCRIPT	VS OPUS 5
DeepSeek V4 Flash Latest	OPENROUTER	0.030	0.100	**$1.29**	$0.0007	177×

MODEL	ROUTE	IN $/M	OUT $/M	CORPUS PASS	PER TRANSCRIPT	VS OPUS 5
DeepSeek V4 Flash 0423 (pinned)	OPENROUTER	0.068	0.168	$2.73	$0.0015	83×
DeepSeek V4 Flash (standard)	OPENROUTER	0.098	0.196	$3.80	$0.0021	60×
Gemini 2.5 Flash-Lite	OPENROUTER	0.100	0.400	$4.49	$0.0025	51×
Gemini 3.1 Flash-Lite	OPENROUTER	0.250	1.500	$12.76	$0.0071	18×
Gemini 3 Flash	OPENROUTER	0.500	3.000	$25.52	$0.0142	9×
Claude Haiku 4.5	ANTHROPIC	1.000	5.000	$45.48	$0.0253	5×
Claude Sonnet 5	ANTHROPIC	2.000	10.000	$90.96	$0.0505	2.5×
Gemini 3.1 Pro	OPENROUTER	2.000	12.000	$102.07	$0.0567	2.2×
Claude Opus 5	ANTHROPIC	5.000	25.000	**$227.40**	$0.1263	–

> One corpus pass on Opus 5 costs more than three years of DeepSeek at a pass every week.

The same comparison at programme scale

A single pass over a single theme is not the real workload. Thematic research means many themes — the glory of God, forgiveness, covenant, discipleship, and so on — each requiring its own full pass over the corpus. At twelve themes the gap stops being a line item and becomes a budget decision:

Twelve-theme research programme

1,800 transcripts × 12 themes = 37,800 calls, 372M input + 34.7M output tokens

MODEL	PROGRAMME COST	COST VS DEEPSEEK
DeepSeek V4 Flash Latest	**$15.44**	–
Gemini 2.5 Flash-Lite	$53.91	+$38

MODEL	PROGRAMME COST	COST VS DEEPSEEK
Gemini 3.1 Flash-Lite	$153.10	+$138
Gemini 3 Flash	$306.20	+$291
Claude Haiku 4.5	$545.75	+$530
Claude Sonnet 5	$1,091.50	+$1,076
Claude Opus 5	$2,728.76	+$2,713

A NOTE ON HONESTY IN THESE FIGURES

These are list prices against a modelled corpus. Three things will move them, and all three move *every* model in the table in the same direction, so the ratios hold: retries on transient failures add perhaps 2–5%; the real corpus may skew longer than modelled (the sensitivity run at +50% length simply doubles every figure); and Gemini's newest Flash tier is currently on introductory pricing that is scheduled to rise on 1 January 2027. DeepSeek's position is not sensitive to any of these.

03

The hidden cost nobody budgets: iteration

The corpus pass is the cheap part. Getting the prompt right is what you actually pay for.

The figures above assume we run the pipeline once and accept the result. That is not how this work goes. Tuning a thematic rubric means running a sample, reading the output, discovering the model is over-scoring tangential mentions, adjusting the prompt, and running again. Ten to thirty iterations before a rubric is trustworthy is normal, not excessive.

On an expensive model, that iteration budget is the thing that quietly kills the project. Every experiment carries a real price, so the team starts rationing experiments — testing on twenty transcripts instead of two hundred, accepting the first prompt that looks acceptable, and shipping a rubric nobody has properly stress-tested. The cost structure begins dictating the research method.

At DeepSeek rates, a full 1,800-transcript run costs less than a cup of coffee. Iteration stops being a budgeted decision and becomes free. We can run the whole corpus twenty times while refining the rubric and still spend under $30. That is not merely a saving — it is a different and better way of doing the research.

Iterating on Claude Opus 5

`20 full-corpus experiments`

- **$4,548** in inference
- Every experiment needs justifying
- Testing shrinks to small samples, so rubric flaws surface late
- Prompt quality is bounded by budget, not by judgement

Iterating on DeepSeek V4 Flash

`20 full-corpus experiments`

- **$25.80** in inference
- No approval needed for any single run
- Full-corpus testing every time, so flaws surface immediately
- Prompt quality is bounded only by how carefully we read the output

04

The subscription we already pay for cannot fund a single line of this

This is the most common and most expensive misunderstanding in any conversation about AI cost. It deserves the fullest explanation in this document, because every budget figure above depends on it being understood.

The natural question from anyone reviewing this proposal is: *we already pay Anthropic every month — why are we being asked to pay for tokens on top of that?*

The answer is that Anthropic sells two entirely different things under the same brand name, on two separate billing systems, through two separate websites, with two separate sets of credentials. Paying for one gives you no access whatsoever to the other. This is not a technicality or a loophole to be negotiated — it is how the products are deliberately structured, and the same structure applies at OpenAI, Google, and every other major vendor.

> THE DISTINCTION IN ONE LINE
>
> **A subscription is a seat for a person. An API key is fuel for a machine.** Our transcript pipeline is a machine. It has no seat, no login, and no human at the keyboard — so there is nothing for the subscription to apply to.

The two products, side by side

It helps to see exactly where the line falls. Note in particular that the two are managed on different domains — `claude.ai` versus `console.anthropic.com` — and that a login for one is not a login for the other:

Claude subscription vs Claude API

Two products, two bills, no overlap between them

	CLAUDE.AI SUBSCRIPTION	CLAUDE API
What you buy	A named person's right to use Anthropic's own apps	Metered compute your own software can call
Managed at	`claude.ai`	`console.anthropic.com`
How you pay	Flat monthly fee per seat	Per million tokens, drawn from prepaid credit
Credential	Email and password — a human login	A secret API key — issued to software
Reached by	Web app, desktop app, mobile app, Claude Code	Any program that can make an HTTPS request
Usage limits	Capped at a human's working pace, resetting on a rolling window	Rate-limited by concurrency and tokens per minute; scales with spend
Output	Text on a screen for a person to read	Structured, schema-validated data for a program to consume
Can it run our pipeline?	**No — at any price tier**	**Yes**

Our own configuration file already carries this warning, written into `.env.example` after the team ran into it directly: *"Needs credits on console.anthropic.com; a Claude.ai subscription does not fund API calls."*

"But Claude Code works with our subscription — isn't that programmatic?"

This is the sharpest version of the objection and it deserves a direct answer, because on the surface it looks like a counter-example. Claude Code is a developer tool that runs in a terminal, writes files, and executes commands — and yes, it can be authorised with a Claude.ai subscription rather than API credit.

But Claude Code is still *an application Anthropic built, that a developer sits in front of*. The subscription entitles a person to use Anthropic's software. It does not turn that entitlement into an endpoint that *our* software is allowed to call. There is no address our pipeline can send a request to that draws on a subscription, and no credential it could present if there were. The subscription is consumed by a human working interactively; it is not a fuel line that other programs can tap.

This is the crux of the whole proposal. What we need is not a better assistant for a person. We need an **engine our own software can call, 3,150 times, unattended, overnight**, and that is a category of product the subscription simply does not contain — not at Pro, not at Max, not at any tier. The moment we want to build something rather than converse with something, we are in API territory and paying per token. The only remaining question is whose per-token rate.

Why the chat window cannot do the job even by brute force

Suppose we accepted the constraint and tried to do it by hand anyway — staff pasting transcripts into the chat window to avoid API charges. It is worth working out what that actually costs, because it is the arrangement organisations drift into when they believe the subscription is already paying for everything.

One theme requires **3,150 separate paste-and-copy operations**, each producing a JSON object a person must then transfer into a results file without a transcription error. At an optimistic two minutes per operation, that is roughly **105 hours of staff time for a single theme**, and about **1,260 hours — most of a full-time year — for the twelve-theme programme**. Against that, the same twelve themes cost $15.44 and run unattended.

And the labour is the smaller problem. The chat window structurally cannot provide what the pipeline depends on:

- **No schema enforcement.** The API constrains the model's decoder so the JSON is valid by construction. A chat reply is prose that merely looks like JSON, and a single malformed field silently corrupts a result.
- **No concurrency.** The pipeline runs up to twelve calls simultaneously. A person works strictly one at a time.
- **No checkpointing.** An interrupted batch resumes from its last completed transcript. An interrupted human loses their place.
- **No usage record.** Every API call returns exact token counts, which is how the figures in this document exist at all. Chat sessions produce no auditable cost or reproducibility record.
- **No model comparison.** The evaluation gate in §09 depends on sending byte-identical input to several models. Retyping a prompt into different chat windows is not a controlled comparison.
- **No integration.** Nothing downstream — the marker extraction, the aggregation, the Word report — can consume text a person reads off a screen.

What this means for the decision

The choice was never “use the subscription we already have” versus “pay for OpenRouter”. That option does not exist. Whichever way we go, this workload is billed per token on top of any subscription we hold, and the subscription continues to be useful for exactly what it is for — staff working interactively with Claude.

Once that is clear, the comparison becomes a straightforward one between per-token rates for the same operation: **$227.40 a pass from Anthropic, or $1.29 from DeepSeek through OpenRouter.** And it clarifies what OpenRouter actually is in this picture. It is not a cheaper subscription. It is the same category of product as the Anthropic API — metered compute, an API key, no seats — bought from a marketplace of vendors instead of from one.

05

Where the value actually sits: the model, or what we build on it

The strategic argument, and the one most likely to be challenged. It deserves a careful version rather than a slogan.

The instinct behind “let's use the best model” is sound in general and wrong here, and it is worth being precise about why rather than simply asserting that all models are the same.

The honest position is this. Frontier models are genuinely better than cheap ones — at long-horizon agentic work, at hard multi-step reasoning, at complex code, at holding a difficult problem across many turns. Anthropic's advantage is real and largely lives in the engineering around the model: tool orchestration, context management, safety behaviour, agent frameworks. Those are excellent products and worth their price *for the problems they solve*.

But the capability distribution is not uniform across tasks. It is wide at the frontier — the hardest agentic and reasoning work — and it narrows sharply as tasks become bounded and well-specified. Our task sits at the narrow end. Score one passage against a rubric; emit a fixed JSON object. Every model in the comparison table supports native structured output, which means the schema is enforced by the decoder rather than hoped for from the prompt. The failure mode we care about is not “can it think” but “does it apply the rubric consistently” — and consistency at constrained scoring is where cheap models have closed the gap fastest.

> **We are not buying intelligence. We are buying a rubric applied 3,150 times without drifting.**

The infrastructure argument, stated carefully

Anthropic's premium buys, among other things, an infrastructure layer: agent loops, memory, managed sandboxes, tool execution. For this project, we do not use any of it. Our infrastructure is our own, and it is specifically shaped to this corpus:

- **Boundary-marker extraction** — the model returns two short phrases; regex slices the real passage. Cuts output tokens by an order of magnitude and removes paraphrasing risk entirely.

- **Adaptive chunking with overlap** — sized to this corpus's transcripts, with map-reduce synthesis, rather than a generic long-context strategy.
- **Append-only JSONL checkpointing** — an interrupted 1,800-transcript batch resumes rather than re-paying for completed work. Chosen over rewriting one document specifically because that is $O(N^2)$ at this scale and leaves a torn-file window on every save.
- **Three-tier concurrency control** — per-chunk, per-transcript, and a global in-flight ceiling, so we tune throughput against a provider's rate limit rather than accepting whatever a hosted agent gives us.
- **Structured Word report generation** — output shaped for the people who will actually read it.

None of that comes from a model vendor. All of it is ours, and it is the part that makes the results useful. What we need from the vendor is one narrow thing done reliably and cheaply: score a passage, return valid JSON. Paying a frontier premium for a hosted infrastructure layer we have replaced with better-fitted code of our own is paying twice for one thing and using neither fully.

> WHERE THE PREMIUM WOULD BE JUSTIFIED
>
> If a future project needs genuine multi-step reasoning across documents, autonomous tool use, or judgement calls with real consequences, the frontier premium may well be correct — and OpenRouter does not prevent that. Claude models are available through it too. The argument is not “never pay for a frontier model”. It is “match the model to the task, and keep the ability to make that choice per task rather than once for everything.”

06

From buying a product to owning a capability

The proposal is not really about DeepSeek, and it is not really about saving $226 a run. It is about which layer of the stack the Foundation occupies.

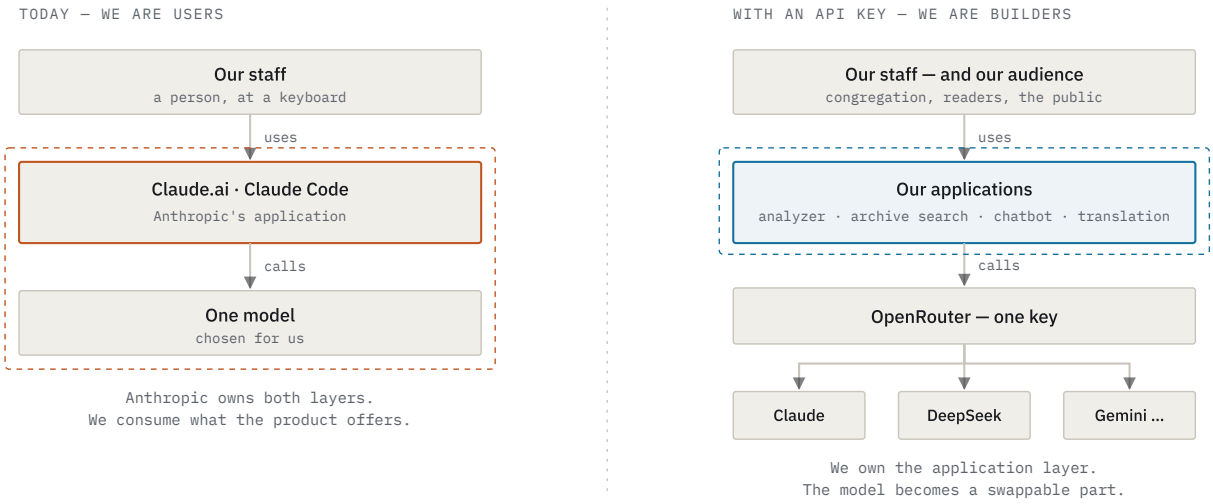
Everything so far has argued about price. This section argues about position, and it is the part with the longest consequences.

Claude.ai is not the model. It is an application built on the model.

This distinction is easy to miss because the branding hides it. When staff use Claude, they are not using a model — they are using a *product* that Anthropic built on top of a model: a chat interface, a conversation history, a set of features, a set of defaults, a set of decisions about what the tool

should and should not do. The model underneath is a component. The product is the thing wrapped around it.

The moment you hold an API key, you are standing where Anthropic stands. The same component they built their product on is available to build *ours* on — and ours can be shaped to this Foundation rather than to the general public. That is what an API key actually buys, and it is worth far more than the discount.



The same model, two positions in the stack. On the left, the application layer belongs to Anthropic and we reach the model only through the product they built on it — so what we can do is bounded by what that product does. On the right, the application layer is ours: we build software fitted to this Foundation, serve it to our audience as well as our staff, and treat the model as a component we can replace without rewriting anything above it. The dashed outlines mark who controls what.

> **The ceiling on what we can build stops being someone else's product roadmap and starts being our own imagination.**

Approving an Anthropic account approves access to Anthropic models. Approving OpenRouter approves access to a catalogue — Anthropic, Google, DeepSeek, OpenAI, Meta, Mistral, xAI and others — behind a single credential, a single balance, and a single invoice. The organisational implications are larger than the per-token saving.

Per-vendor accounts

Current arrangement

- A separate account, key, invoice and finance approval per vendor
- Trying a new model is a procurement request
- A vendor outage or price rise stops the work
- Model choice is fixed at approval time, not at task time
- Spend is scattered across bills nobody consolidates

One routed account

Proposed

- One key, one balance, one invoice, one approval
- Trying a new model is a one-line config change
- Automatic failover across providers serving the same model
- Model choice made per task, revisited whenever the market moves
- Every call attributed and visible in one dashboard

What that actually makes buildable

The transcript analyzer is one application. It is not a special case — it is simply the first thing we happened to build. The same key, the same balance and the same approval cover everything below, and the archive we are about to index is the raw material for most of it.

Costs shown are monthly, at DeepSeek rates, sized to the Foundation's realistic volumes: roughly 60 new teachings a month and a few thousand audience interactions.

Search and discovery over our own archive

`~$0.60 / month`

- **Meaning-based archive search.** “Where has this been taught on suffering as formation?” — answered by concept, not keyword match.
- **A grounded question-answering assistant.** Answers drawn only from our published teaching, with citations back to the transcript and timestamp.
- **Related-teaching suggestions.** Surfacing the other places a subject was treated, so nothing sits orphaned in the archive.
- **Doctrinal consistency checks.** Flagging where the same subject was treated differently across years, for a human to review.

Publishing and production

`~$0.15 / month`

- **Titles, summaries and descriptions** generated for each new teaching, in a consistent house voice.
- **Chapter markers and section headings** for long recordings, so listeners can navigate.
- **Transcript cleanup.** Correcting raw speech-to-text — punctuation, speaker labels, proper nouns — before it enters the corpus.
- **Pull-quote and clip selection.** Identifying the passages worth excerpting for social and print.

Reach, translation and accessibility

`~$0.60 / month`

- **Translation into the languages our audiences actually use** — four languages across every new teaching costs cents, where human translation is priced per thousand words.
- **Subtitles and captions** generated from the transcripts we already hold.
- **Plain-language versions** of dense material, for younger readers and second-language audiences.
- **Study guides and discussion questions** derived from a teaching, for small groups.

Member care and communications

`~$0.15 / month`

- **First-line answers to common questions** — service times, giving, events — freeing staff for the conversations that need a person.
- **Triage and routing** of incoming requests to the right person, with urgent items escalated rather than answered.
- **Correspondence drafting** in the Foundation's voice, always reviewed and sent by a human.
- **Daily devotional drafts** assembled from the archive, edited before publication.

Internal operations

`~$0.10 / month`

- **Meeting notes and action items** from recorded internal sessions.
- **Grant and report drafting**, working from our own documents and figures.
- **Document classification and filing** across years of unsorted material.
- **Data extraction** from scanned forms, registers and historical records.

Research and scholarship

`~$1.29 per corpus pass`

- **The thematic analysis programme** this proposal funds — the first application, not the last.
- **Longitudinal study.** How the treatment of a theme developed across two decades of teaching.
- **Scripture and citation indexing** across the whole corpus, built once and reused everywhere.
- **Comparative analysis** against other published bodies of work.

THE WHOLE CATALOGUE, COSTED

Every application above, running every month, comes to **under $2 a month** in inference at DeepSeek rates — less than one Claude Opus corpus pass would cost for a decade. The constraint on this list has never been the money. It is that each new idea currently requires its own vendor decision, its own account and its own approval, and that friction is what stops small useful projects from ever being attempted. One approved key removes it permanently.

There is a second-order effect worth naming. Because every one of these applications is built on *our* infrastructure rather than inside someone else's product, they compound. The chunking, the checkpointing, the marker extraction and the report generation written for the analyzer are not single-use — they are the beginnings of a toolkit the next application inherits. Each thing we build makes the following one cheaper. Nothing built inside a chat window compounds that way, because nothing built there is ours to reuse.

HOW OPENROUTER MAKES ITS MONEY

Worth stating for the finance review, because it is unusually simple: OpenRouter does not mark up token prices. Inference is billed at the upstream provider's list rate. Their revenue is a **5.5% fee on credit top-ups** by card (5% by crypto), with a $0.80 per-transaction minimum. That minimum means small top-ups carry a poor effective rate — on a $5 top-up it works out near 16% — so we should fund the account in reasonable increments. On a $200 top-up the fee is $11. There is no subscription and no monthly commitment; unused credit simply remains as credit.

07

What switching actually requires

This is the part of the proposal with no engineering risk attached, because the work was already done.

The analyzer was built provider-neutral from the start. It talks to any OpenAI-compatible endpoint through `base_url` and `api_key` alone — `client.py` is a thin factory over the standard SDK, and nothing above it knows or cares which vendor is on the other end. Switching provider is editing three lines of `.env`:

The entire migration

Configuration only — no code changes, no dependency changes, no schema changes

SETTING	ANTHROPIC DIRECT	OPENROUTER
`LLM_PROVIDER`	`anthropic`	`openrouter`
`LLM_BASE_URL`	`api.anthropic.com/v1/`	`openrouter.ai/api/v1`
`LLM_DEFAULT_MODEL`	`claude-sonnet-5`	`deepseek/deepseek-v4-flash`

Both configurations are already written into `.env.example` as documented, commented alternatives. The repository has in fact already been switched (commit `43442cd`, *"switched to openrouter"*), which means this proposal is asking leadership to ratify and fund a technical decision that has been validated in practice, not to authorise a risky rewrite.

Reversal is symmetric. If DeepSeek disappoints on evaluation, the same three lines point back at Claude — and can point at Claude *through OpenRouter*, so even choosing Anthropic models does not require abandoning the routed arrangement.

08

Risks, and what we do about each

Stated in full, including the ones that argue against this proposal.

A proposal that lists only advantages should not be trusted. These are the genuine objections, with the mitigation for each.

The cheaper model might score less accurately than Claude

This is the real risk, and it should not be waved away — it should be measured. The repository already contains the instrument: `transcript_theme_analyzer/eval/compare_models.py` runs the same transcript and theme across several models and prints score, token usage and latency side by side.

Mitigation: before committing to a full run, take a stratified sample of 30–50 transcripts with known thematic content, score them on Claude Opus 5, DeepSeek V4 Flash and a Gemini Flash tier, and have a domain reader blind-rank the outputs. Adopt the cheapest model whose agreement with the expert reading is acceptable. At $1.29 per full pass, running the definitive comparison costs almost nothing, so this decision can be made on evidence instead of on brand.

Routing through an intermediary adds a dependency and a point of failure

True. OpenRouter sits between us and the model provider, so its availability becomes our availability.

Mitigation: the same provider-neutral design that makes switching to OpenRouter a three-line change makes switching away from it a three-line change. We are never locked in, and the pipeline's checkpointing means an outage mid-run costs the in-flight transcripts only, not the batch. In practice routing also *adds* resilience, because OpenRouter fails over between multiple upstream providers serving the same model — something a direct single-vendor account cannot do.

Data handling and where transcripts are processed

Transcript content is sent to a third-party inference provider, and the routed arrangement means the specific upstream provider may vary.

Mitigation: this consideration exists identically with a direct Anthropic account — the content leaves our infrastructure either way. What changes is that it becomes a decision we make explicitly. OpenRouter allows pinning to specific upstream providers where that matters. Before the full run, confirm the material's sensitivity classification: published teaching intended for wide distribution carries a different profile from pastoral or private recordings, and if the corpus contains the latter it should be segregated and handled under a stricter policy regardless of vendor.

A moving model target makes results non-comparable across a long batch

A model alias such as `deepseek-v4-flash` redirects to whatever the latest version is. If it updates mid-batch, transcripts analysed on different days may not be scored by the same model — a genuine methodological problem for research output.

Mitigation: use a pinned snapshot for any batch whose results will be compared or published. `deepseek/deepseek-v4-flash-0423` is a fixed version at $2.73 per corpus pass — a $1.44 premium over the floating alias, which is a negligible price for methodological consistency. Our `.env.example` already documents this option and the reason for it.

Prepaid credit is a different budgeting model than invoiced billing

OpenRouter is prepaid: credit is bought up front and drawn down. Some finance processes prefer arrears invoicing.

Mitigation: at these volumes prepayment is an advantage, not a constraint — it is a hard spending ceiling that cannot be exceeded by a runaway job, which invoiced billing does not give us. A $50 balance covers the entire proposed first-year programme with substantial headroom. Fund in increments of $100 or more so the $0.80 minimum transaction fee stays negligible.

Cheap model pricing may not last

Today's rates reflect intense competition. Gemini's newest Flash tier is explicitly on introductory pricing scheduled to rise on 1 January 2027, and others may follow.

Mitigation: this is precisely the argument *for* a routed arrangement rather than against it. When one provider's price rises, we move to another by editing one line, without a new vendor account or a new approval cycle. A direct single-vendor account gives us no such option — it converts every price change into a procurement project.

09

Recommendation

What we are asking leadership to approve, and what happens next.

Adopt OpenRouter as the Foundation's inference provider, and run the transcript analysis programme on DeepSeek V4 Flash, subject to an evaluation gate.

Proposed budget, first year

Covering the full thematic programme, iteration, and the Foundation's other AI work

ITEM	COST
Model evaluation on a 50-transcript sample, across four models	< $1
Prompt iteration — 20 full-corpus experiments	$25.80
Twelve-theme research programme, pinned snapshot model	$32.81
Contingency for retries, re-runs and corpus growth (50%)	$29.31
Headroom for other Foundation AI applications	$60.00
Recommended initial credit purchase	**$150.00**
The same programme on Claude Opus 5, for comparison	$7,300+

A single $150 credit purchase — less than the cost of one full corpus pass on Opus 5 — funds the entire first-year programme with roughly four times the headroom it needs, and leaves capacity for every other AI application the Foundation might want to try.

Proposed sequence

01 **Fund the account**

Purchase $150 in OpenRouter credit. Set a hard spend ceiling on the key so no job can exceed the approved amount.

02 **Run the evaluation gate**

Score a stratified 50-transcript sample on Claude Opus 5, DeepSeek V4 Flash and Gemini Flash-Lite using the existing `compare_models` harness. Have a domain reader blind-rank the outputs.

03 **Decide on evidence, and record the decision**

Adopt the cheapest model whose agreement with the expert reading is acceptable. If that turns out to be Claude, we run Claude — through OpenRouter, keeping the routing benefits. Document the comparison so the choice can be revisited when the market moves.

04 **Pin the model version**

Switch to a pinned snapshot for the production batch, so every transcript in a published result set was scored by the same model.

05 **Run the corpus**

Execute the full 1,800-transcript pass per theme, with checkpointing enabled so any interruption resumes rather than re-paying.

06 **Report actuals against this estimate**

Publish real token usage and spend from the first full run, so the next budget request rests on measured figures rather than a model.

> THE DECISION IN ONE SENTENCE
>
> We have already built infrastructure well-fitted to this corpus; what remains is to stop paying frontier-tier rates for the one narrow, commoditised operation we actually need from a vendor, and to buy that operation from a market rather than from a single supplier — keeping, in the process, the freedom to change our minds cheaply.

Sources and reproducibility. Token volumes are derived from recorded usage in `results/x-ai__grok-4.3.json` and from the pipeline's own chunking logic. All cost figures are produced by `cost_model.py`, which can be re-run against the real corpus with `--transcript-dir` for exact rather than modelled numbers. Anthropic rates are first-party API list prices; DeepSeek and Gemini rates are OpenRouter list prices, all retrieved 27 August 2026. OpenRouter fee structure per their published pricing terms. Prices in USD and subject to change — the modelled ratios are more durable than the absolute figures.

Prepared for internal review · GLH Extraction Team · 27 August 2026